laborch: Package Manual

Version 0.2.0 (Phase 1)

LabOrch project

November 2026

Contents

1. Concepts and vocabulary 1
2. Installation 2
3. Quick start 2
4. Package layout 2
5. Module reference 3
6. Recipes 11
7. Data conventions 12
8. Testing 12
9. Known limitations and roadmap 13

`laborch` is the Python core of LabOrch, a computer-assisted orchestration tool. Given a **Target** sound and an **Ensemble** of instruments, it finds the combination of recorded notes whose summed spectrum best matches the Target. The package gathers the logic of the 2019–2020 notebooks (1.5, 1.7, 1.11, 1.12 and the `tool*.py` widget scripts) into tested, importable modules (Phase 0), and turns the Bank into a dataset (Phase 1): **Bank v2** holds power spectra averaged over each note's automatically detected sustain, keeps every recording's level, and comes with a metadata table. See `docs/bank_v2.md` for how it was made and what it covers.

1. Concepts and vocabulary

Term	Meaning	In code
Bank	Every recorded note of every instrument (1,515 sounds, 33 instruments), each stored as an averaged spectrum	`Bank`
Ensemble	The instruments the composer writes for, e.g. a string quartet	a list of instrument codes, `["vln", "vla", "vlc"]`
Palette	Every Bank sound the Ensemble can play (notes, modes, dynamics), as a matrix of spectra	`Palette` (from `Bank.select`)
Target	The sound to imitate: a Bank sound or an uploaded recording	a 2,048-element NumPy array
Recommendation	The weighted set of Palette sounds that best reproduces the Target	`Fit`

The model (Rose & Hetrick 2009, "RH2009") treats the Target spectrum T as a weighted sum of Palette spectra X_j:

$$T(f_i) \approx \sum_j X_{ij}\,\alpha_j, \qquad \alpha_j \geq 0$$

The weights α are found by non-negative least squares (NNLS), optionally weighted per frequency bin by $1/\sigma_i$ as in *Numerical Recipes* §15.4.

In Bank v2 the spectra are **power** spectra at each recording's level. Powers of uncorrelated sounds add, so the model is exact for a mix, and a weight α_j is a power gain on recording j: $\alpha_j = 1$ means "as recorded", $\alpha_j = 0.5$ means 3 dB quieter. (In Bank v1 the spectra were unit-sum magnitudes and a weight was only a share of the fit.)

2. Installation

Requirements: Python $\geq$ 3.9, NumPy, SciPy. Audio features (reading recordings, splitting, mixing) also need **pydub** and **ffmpeg** on the `PATH`.

From the `LabOrch` folder:

```
pip install -e ".[dev]"     # package + pydub + pytest, editable
# or, without installing:
#   import sys; sys.path.insert(0, "path/to/LabOrch")
```

On Windows, ffmpeg can be installed with `winget install ffmpeg`.

The package finds the data relative to its own location (the repository root). To use data elsewhere, set the environment variable `LABORCH_ROOT` before importing (see 5.1).

3. Quick start

```
from laborch import Bank, nnls_fit

bank = Bank.load()                      # Bank v2: power spectra + metadata
target_name = "trb-f2-ord-mf"           # trombone F2, mf
target = bank.spectrum(target_name)

palette = bank.select(instruments=["clb", "bcl"],   # Ensemble: clarinets
                      exclude=[target_name])
fit = nnls_fit(palette, target)

print(fit.summary(4))
# nnls: 62 sounds, relative error 0.644
#     0.2184  clb-c5-ord-mf
#     0.2065  clb-f4-ord-mf
#     0.1714  clb-g4-ord-mf
#     0.1095  clb-b4-ord-mf

best3 = fit.top(3, refit=True)          # keep 3 sounds, re-solve on them
best3.ranked()                          # [('clb-f4-ord-mf', 0.346), ...]

bank.meta[target_name]["level_db"]      # -26.3 dBFS; the full metadata row

from laborch.audio import render_fit    # needs pydub + ffmpeg
render_fit(bank, best3, n=3, path="trb_from_clarinets.wav")
```

With every bin weighted equally ($\sigma = 1$), power spectra let the strongest partials dominate, so the clarinets imitate the trombone's loud upper partials rather than its fundamental. Weighting

by the Target's own spectrum (`sigma=np.maximum(target, target.max()*1e-4)`, relative error per bin) brings `bcl-f2` and `bcl-f3` back into the top four. Choosing σ is Phase 2. With `Bank.load(version=1)` the example gives the Phase 0 answer (`bcl-f2-ord-mf` first).

The notebook `Notebooks/2.0_laborch_quickstart.ipynb` walks through the same example with plots and playback.

4. Package layout

```
LabOrch/
├── laborch/
│   ├── __init__.py   public API (re-exports)
│   ├── config.py     paths and FFT constants
│   ├── bank.py       sound names, Bank (v2 and v1), Palette
│   ├── metadata.py   the metadata table: families, envelopes, ranges
│   ├── build.py      rebuild Bank v2 from the recordings
│   ├── spectra.py    audio loading, sustain detection, power and v1 spectra
│   ├── solve.py      DOT, pseudo-inverse and NNLS decompositions; Fit
│   ├── audio.py      gains and mixing for playback
│   └── synth.py      synthetic tones, spectrum-to-audio resynthesis
├── tests/            pytest suite (61 tests)
├── docs/             this manual, bank_v2.md report, make_bank_report.py
└── pyproject.toml
```

Names importable directly from `laborch`:

`Bank`, `Palette`, `SoundName`, `parse_name`, `INSTRUMENT_NAMES`, `Fit`, `dot`, `nnls_fit`, `pinv_fit`, `chi2`, `make_fft`, `power_spectrum`, `find_sustain`, `spectral_peaks`, `FAMILIES`, `tone`, `spectrum_to_audio`.

Audio helpers are imported from their module (`laborch.audio`, `laborch.spectra`) so that the core works without pydub.

---

5. Module reference

5.1 `laborch.config`

Constants only; no functions. Change paths here, not in calling code.

Name	Value	Meaning
`ROOT`	repository folder, or `$LABORCH_ROOT`	base for every path below
`BANK_DIR`	`ROOT/Soundbank_McGill`	McGill Bank folder
`FFT_DIR`	`BANK_DIR/FFTBank`	current spectra, one `.txt` per sound
`SOUND_DIRS`	`Soundbank_WB`, `Soundbank_Str`, `Soundbank_PianoPercSax`	the `.aif` recordings
`OLD_FFT_DIR`	`ROOT/OldFFTBank`	2009 `.nbb` spectra (reference)
`MISC_SOUNDS_DIR`	`ROOT/MiscSounds`	extra Targets (trumpet mutes, multiphonics)
`CACHE_FILE`	`ROOT/.cache/fftbank.npz`	binary cache of Bank v1
`BANK_VERSION`	2	Bank loaded by default
`BANK_FILE`	`BANK_DIR/Bank_v2/bank_v2.npz`	Bank v2 names and power spectra (float32)
`BANK_META`	`BANK_DIR/Bank_v2/bank_v2.csv`	Bank v2 metadata table
`BUILD_CACHE`	`ROOT/.cache/build_v2.npz`	resumable state of `laborch.build`

Name	Value	Meaning
`SAMPLE_RATE`	44100	Hz
`NFFT`	4096	Hamming window length (samples)
`HOP`	512	step between windows (samples)
`NBINS`	2048	bins kept per spectrum
`BIN_HZ`	10.766…	Hz per bin (`SAMPLE_RATE / NFFT`)
`SHAVE`	0.15	v1 (and fallback): fraction trimmed from each end of a note
`SMOOTH_FRAMES`	17	v2: level smoothed over 17 frames (≈ 0.2 s) before finding the sustain
`LOUD_DB`	10	v2: reference level = median of frames within 10 dB of the peak
`SUSTAIN_DB`	4	v2: sustain = frames within ±4 dB of the reference
`EDGE_FRAMES`	2	v2: frames dropped at each end of the sustain
`MIN_FRAMES`	3	v2: fewer frames than this → fixed-trim fallback
`DECAY_DB`, `DECAY_MAX_S`	20, 0.5	v2, struck sounds: window from the peak until 20 dB down, at most 0.5 s

`LABORCH_ROOT` is read once, at import time.

5.2 `laborch.bank`

Constants **`INSTRUMENT_NAMES`** — `dict[str, str]`, the 33 instrument codes of the current Bank mapped to names, e.g. `"vln"` → `"Violin"`, `"clb"` → `"Bb Clarinet"`, `"tpc"` → `"C Trumpet"`.

`MODE_NAMES` — `dict[str, str]`, performance-mode codes: `ord` ordinary (winds: with vibrato), `mtd` muted, `nhr` natural harmonic, `ahr` artificial harmonic, `hmo` harmon mute (open), `stm` straight mute, `ped` pedal tone, `bwd` bowed, `hdm` hard mallet, `bbt` brass beater. Confirmed against François' Instrument Index spreadsheet.

`VARIANT_NAMES` — `o` open string, `x` stopped (the same pitch fingered on the next string down). Variants occur only on open-string pitches.

`parse_name(name) -> SoundName` Parses a Bank sound name, with or without extension.

```
>>> s = parse_name("ctb-2fs4-nhr-mf.txt")
>>> s.instrument, s.string, s.pitch, s.mode, s.dynamic
('ctb', 2, 'fs4', 'nhr', 'mf')
```

Raises `ValueError` for names that do not follow the scheme.

`class SoundName` (frozen dataclass)

Field	Type	Example
`instrument`	`str`	`"vln"`
`note`	`str`	`"a"`, `"cs"` (sharps as `s`)
`octave`	`int`	`4`

Field	Type	Example
`mode`	`str`	`"ord"`
`dynamic`	`str`	`"pp"`, `"mf"`, `"ff"`
`string`	`int` or `None`	`2` in `ctb-2fs4-nhr-mf`
`variant`	`str`	`"o"` (open string) / `"x"` (stopped) in `vln-a4o-ord-mf`

Property	Returns
`pitch`	`"a4"`
`midi`	MIDI number, C4 = 60, A4 = 69
`freq`	equal-tempered fundamental in Hz, A4 = 440
`str(s)`	the original name

`class Palette` (dataclass)

Attribute	Type	Meaning
`names`	`list[str]`	sound names, one per column
`matrix`	`ndarray (2048, n)`	spectra as **columns** (the basis X of RH2009)

`len(palette)` is the number of sounds. Palettes are normally made with `Bank.select`, but any set of spectra can be wrapped: `Palette(["a", "b"], np.column_stack([spec_a, spec_b]))`.

`class Bank` All Bank spectra, one **row** per sound, rows sorted by name.

Constructor `Bank(names, spectra, source="", kind="magnitude", version=1, meta=None)` — `spectra` must have shape `(len(names), 2048)`; every name must parse with `parse_name`; `kind` is `"power"` or `"magnitude"`.

Class methods

Method	Description
`Bank.load(version=2, normalize=None, **kw)`	Loads Bank v2 (default) or v1. `normalize="unit"` divides every spectrum by its sum, keeping only its shape. Other keywords go to `load_v2` / `load_v1`.
`Bank.load_v2(path=BANK_FILE, meta_path=BANK_META)`	Bank v2: power spectra at recorded level (`kind="power"`) and the metadata table. Raises `FileNotFoundError` if `bank_v2.npz` is missing (build it with `python -m laborch.build`).
`Bank.load_v1(fft_dir=FFT_DIR, use_cache=True, cache_file=CACHE_FILE)`	Bank v1 from `FFTBank/*.txt` (unit-sum magnitudes). The first call reads 1,515 text files (about a minute on Dropbox) and writes `.cache/fftbank.npz`; later calls load the cache in about a second.
`Bank.load_old(nbb_dir=OLD_FFT_DIR)`	Loads the 2009 `.nbb` spectra (2,048 float32 values each). One malformed name (`obo-f5-c6.nbb`) is skipped with a warning. Old instrument codes differ (`cla`, `hrn`, `tpt`, `mar`, `btr`).

Attributes

Attribute	Meaning
`names`	sorted list of sound names
`spectra`	`ndarray (n_sounds, 2048)`
`info`	`dict[name, SoundName]`
`source`	file or folder the Bank was read from
`kind`	`"power"` (v2) or `"magnitude"` (v1, 2009)
`version`	2 or 1
`meta`	`dict[name, dict]`, the metadata row of each sound (v2; empty for v1)
`instruments`	sorted list of instrument codes present (property)

Methods

Method	Returns	Description
`len(bank)`	`int`	number of sounds
`name in bank`	`bool`	membership
`spectrum(name)`	`ndarray (2048,)`	one sound's spectrum (a view; copy before modifying)
`select(instruments=None, modes=None, dynamics=None, midi_range=None, exclude=(), families=None)`	`Palette`	builds a Palette. Each filter is optional and takes a list or a single string; `midi_range=(lo, hi)` is inclusive; `exclude` removes named sounds, e.g. the Target; `families` selects by family (see 5.7) and combines with `instruments` as an intersection. Raises `ValueError` if nothing matches.
`level_db(name)`	`float`	10 $\log_{10}$ of the spectrum's sum: the sound's level in dBFS for v2
`sound_path(name)`	`Path`	the `.aif` recording for a sound; `FileNotFoundError` if absent

```
quartet = bank.select(instruments=["vln", "vla", "vlc"],
                      modes=["ord", "mtd"],
                      midi_range=(48, 84))
```

`frequencies() -> ndarray` Centre frequency in Hz of each of the 2,048 bins (bin 0 = DC). Use it as the x-axis when plotting spectra.

5.3 `laborch.spectra`

The averaged-spectrum pipelines used to build the Banks, plus audio I/O. Functions marked † need pydub and ffmpeg.

Bank v2: `power_spectrum(x, envelope="sustained", region=None) -> (ndarray, Sustain)` The Bank v2 recipe:

1. cut `x` (mono, scaled to −1..1; see `load_float`) into 4,096-sample Hamming frames at hop 512;
2. take each frame's one-sided power spectrum, scaled so it sums to the frame's mean square (Parseval);
3. choose the frames to use with `find_sustain` (or the `region (start_s, end_s)` you give);
4. average the frame powers.

`10*log10(spec.sum())` is the level of the region in dBFS, and spectra of uncorrelated sounds add. Returns the spectrum and a `Sustain`.

```
from laborch.spectra import power_of_file
target, sus = power_of_file("my_target.wav")      # a Target in Bank v2 units
sus.start_s, sus.end_s, sus.level_db, sus.method
```

`find_sustain(ms, envelope="sustained") -> Sustain` Chooses frames from their mean-square levels `ms` (from `frame_power`).

- `"sustained"` (bowed, blown): smooth the dB level over `SMOOTH_FRAMES` (≈ 0.2 s, so vibrato and tremolo do not break it up); take as reference the median of frames within `LOUD_DB` of the smoothed peak; keep the longest run within ±`SUSTAIN_DB` of it, less `EDGE_FRAMES` at each end. Fewer than `MIN_FRAMES` → the v1 fixed 15% trim (`method="trim"`).
- `"decaying"` (struck): from the loudest frame until `DECAY_DB` below it, at most `DECAY_MAX_S`.

Which envelope a Bank sound has is in its metadata (`metadata.envelope_class`).

`class Sustain` (dataclass) `first`, `last` (frame indices, inclusive), `method` (`"sustain"`, `"decay"`, `"trim"`, `"given"`), `level_db` (mean-square level of the region, dBFS), `peak_db` (loudest frame); properties `n_frames`, `start_s`, `end_s` (from the start of the first window to the end of the last).

`power_of_file(path, envelope="sustained", region=None)` †, `load_float(path)` †, `segment_float(segment)` † `power_spectrum` of a file; mono samples scaled to −1..1.

`frame_power(x) -> (P, ms)` and `frames(x)` Per-frame power spectra `(n_frames, 2048)` and mean squares; the frame matrix.

Bank v1

`make_fft(samples, shave=0.15, normalize=True) -> ndarray` The Bank's spectrum recipe (notebook 1.11):

1. drop the first and last `shave` fraction of the samples (attack and decay);
2. slide a 4,096-sample Hamming window in steps of 512;
3. combine the frames as $\left(\sum_{\text{frames}} |X_k|^2\right)^{1/2}/n$, keeping bins 0–2047;
4. if `normalize`, divide by the sum (unit area), as every Bank file is.

`samples` is a 1-D array of mono samples at 44.1 kHz. Raises `ValueError` if the sound is shorter than one window after trimming. Use `normalize=False` to keep the absolute level.

`fft_of_file(path, **kw) -> ndarray` † `make_fft(load_samples(path), **kw)`. Recomputing a Bank file this way matches the stored spectrum to about 0.1% of its peak.

`load_segment(path) -> pydub.AudioSegment` † Reads any ffmpeg-readable file (AIF, WAV, MP3, …).

`segment_samples(segment) -> ndarray` † Mono samples of an `AudioSegment` as floats in the file's integer scale (the scale used to build the Bank). Warns if the sample rate is not 44.1 kHz.

`load_samples(path) -> ndarray` † `segment_samples(load_segment(path))`.

`split_on_silence(path, min_silence_len=500, silence_thresh=-50, keep_silence=100) -> list` † Splits a multi-note recording into one `AudioSegment` per note (notebooks 1.8, 1.9, 1.12). Times in ms, threshold in dBFS.

```
from laborch.spectra import split_on_silence, segment_samples, make_fft
notes = split_on_silence("MiscSounds/Trumpet-mutes_Basson-multiphics/11 Track 11.aiff")
cup_c5 = make_fft(segment_samples(notes[2]))
```

`spectral_peaks(spec, rel_height=0.02, max_hz=None) -> list` Local maxima of a spectrum as `[(Hz, height relative to the tallest), ...]`, lowest frequency first. Peaks below `rel_height` × the tallest, or above `max_hz`, are dropped.

```
>>> spectral_peaks(make_fft(tone(130.81, [1, 4, 10], [1, .5, .25])), max_hz=3000)
[(129.2, 1.0), (527.6, 0.45), (1302.8, 0.21)]
```

5.4 `laborch.solve`

`nnls_fit(palette, target, sigma=None, maxiter=None) -> Fit` Non-negative least squares (Lawson–Hanson, via `scipy.optimize.nnls`):

$$\min_{\alpha} \sum_{i} \left(\frac{T_i - \sum_j X_{ij}\,\alpha_j}{\sigma_i} \right)^2 \quad \text{subject to} \quad \alpha_j \geq 0$$

`sigma` is an optional array of 2,048 positive per-bin uncertainties; `None` weights every bin equally (the 2020 behaviour). This is the main solver.

`pinv_fit(palette, target, sigma=None) -> Fit` Unconstrained least squares via the pseudo-inverse — RH2009's "SVD" method. Weights can be negative (an instrument "subtracting" sound), so use it only for comparison.

`dot(palette, target, normalize=False) -> Fit` RH2009's "DOT" method: the projection $p_j = \sum_i T_i X_{ij}$ of the Target on each sound alone. With `normalize=True` it is cosine similarity (shape only). The weights are scores, not mixing amounts; use DOT to rank or prefilter, not to build a mix.

`chi2(target, model, sigma=None) -> float` Sum of squared, optionally σ-weighted, residuals.

`class Fit` (dataclass) The result of any solver: a Recommendation before playability rules.

Attribute	Meaning
`names`	Palette sound names
`weights`	`ndarray`, one weight per name (same order)
`target`	the Target spectrum
`matrix`	the Palette matrix used
`method`	`"nnls"`, `"pinv"`, `"dot"`, `"dot-cos"`, or `"nnls/top3"` etc.
`sigma`	the σ used, or `None`

Property	Returns
`model`	fitted spectrum `matrix @ weights`

Property	Returns
`chi2`	χ^2 of the fit (σ-weighted if σ was given)
`relative_error`	$\|T - \text{model}\|/\|T\|$, unweighted; 0 is perfect
`order`	indices sorted by weight, largest first

Method	Returns	Description
`ranked(n=None, nonzero=True, rtol=0.0)`	`list[(name, weight)]`	largest first; with `nonzero`, drops weights $\leq$ `rtol` $\times$ `max` (use `rtol=1e-9` to hide round-off)
`top(n, refit=False)`	`Fit`	keeps the n largest weights. `refit=False` keeps their values (as the 2020 mock-up did); `refit=True` re-solves NNLS on those n sounds, which never increases χ^2
`kept_fraction(n)`	`float`	share of total weight in the top n
`summary(n=10)`	`str`	readable report

5.5 `laborch.audio`

Needs pydub and ffmpeg (except `weights_to_db`).

`weights_to_db(weights, floor_db=-60.0, kind="magnitude", relative=True) -> ndarray` Gain in dB of each weight: 20 $\log_{10}$ w for magnitude spectra (v1), 10 $\log_{10}$ w for power spectra (v2). With `relative=True` the largest weight is 0 dB; with `relative=False` the gain is relative to each recording's own level, which is what a v2 weight means. Zero weights get `floor_db`.

`mix(bank, names, gains_db) -> AudioSegment` Overlays the named Bank recordings, each shifted by its gain in dB. The result is as long as the longest sound.

`render_fit(bank, fit, n=8, path=None, headroom_db=0.0) -> AudioSegment` Mixes the top-n sounds of a `Fit`; writes a file if `path` is given (format from its extension, e.g. `.wav`, `.mp3`). In Jupyter, the returned segment plays inline. With Bank v2 the gains are absolute, 10 $\log_{10}$ w on each recording; if the loudest would exceed `headroom_db`, all gains are lowered together, keeping the balance. With Bank v1 the gains are relative to the largest weight.

5.6 `laborch.synth`

Synthetic sounds (notebooks 1.2 and 2.1). Pure NumPy: no pydub needed.

`tone(f0, partials=(1,), amps=None, seconds=1.0, sr=44100, decay=0.0, phases=None, peak=1.0) -> ndarray` A sum of sine partials at multiples of `f0`:

$$x(t) = e^{-\lambda t} \sum_k a_k \sin(2\pi\, p_k f_0\, t + \phi_k)\,, \qquad \lambda = \texttt{decay}$$

Parameter	Meaning
`f0`	fundamental in Hz; `parse_name("mrb-c3-ord-mf").freq` gives 130.81
`partials`	multiples of `f0`; need not be integers (e.g. 3.9 for a real bar)
`amps`	amplitude of each partial (default all 1)
`seconds, sr`	duration and sample rate
`decay`	exponential decay rate in 1/s; 0 = steady
`phases`	starting phase of each partial in radians (default 0)
`peak`	largest absolute sample after scaling; `2**15 - 1` gives 16-bit scale

Raises `ValueError` if the lists differ in length or a partial reaches the Nyquist frequency (22,050 Hz).

`spectrum_to_audio(spec, seconds=1.0, sr=44100, seed=0, threshold=1e-4, peak=1.0) -> ndarray` Additive resynthesis of a magnitude spectrum (for a v2 power spectrum, pass `np.sqrt(spec)`). Every bin k above `threshold` $\times$ the maximum becomes a sine at $k \times 10.77$ Hz with amplitude `spec[k]` and a random phase (`seed` makes it repeatable). Works on Bank spectra, `Fit.model` and `make_fft` output.

Because the Bank stores magnitudes only, the phases are invented: the result keeps the timbre (which partials, how strong) but not the original waveform, attack or decay. A round trip spectrum $\rightarrow$ audio $\rightarrow$ spectrum keeps the shape with a cosine similarity of about 0.97.

```
from laborch.synth import tone, spectrum_to_audio
from IPython.display import Audio
x = tone(130.81, partials=[1, 4, 10], amps=[1, 0.5, 0.25])
Audio(spectrum_to_audio(make_fft(x)), rate=44100)
```

5.7 `laborch.metadata` and `laborch.build`

The metadata table `Soundbank_McGill/Bank_v2/bank_v2.csv`, one row per sound, also available as `bank.meta[name]`. Readable without laborch (spreadsheet, pandas).

Column	Meaning
`name, instrument, instrument_name, family`	sound and instrument; family from `FAMILIES`
`envelope`	`sustained` or `decaying` (struck: piano, marimba, xylophone, glockenspiel, crotales, tubular bells, hard-mallet vibraphone)
`note, octave, pitch, midi, freq_hz`	pitch; `freq_hz` equal-tempered, A4 = 440
`string, variant, variant_name`	string number; `o` open / `x` stopped
`mode, mode_name, dynamic`	performance mode and dynamic
`sub_bank, aif`	McGill sub-bank and the recording's path relative to `ROOT`
`duration_s, sustain_start_s, sustain_end_s, n_frames, method`	the note and the analysed region
`level_db, peak_db`	mean-square level of the region and loudest frame, dBFS
`similarity_v1`	cosine between √(v2 power) and the v1 spectrum: low = the region changed a lot

laborch.metadata

Name	Description
`FAMILIES`	`dict[family, list of codes]`: strings, flutes, double reeds, clarinets, saxophones, brass, keyboard, percussion
`FAMILY_OF`	code → family
`envelope_class(instrument, mode)`	`"sustained"` or `"decaying"`
`describe(name)`	the name-derived columns of a row
`read_table(path=BANK_META)`, `write_table(rows, path=BANK_META)`	CSV I/O, numbers converted
`ranges(rows)`	`(instrument, mode) → (lowest MIDI, highest MIDI)`; Phase 2 uses these as Palette filters
`spreadsheet_names(folder)`	names listed in François' spreadsheets (needs openpyxl)

```
from laborch.metadata import ranges
ranges(bank.meta.values())[("vln", "ord")]     # (55, 96): g3-c7
```

```
python -m laborch.build            # resumable: rerun until it says it wrote the files
python -m laborch.build --fresh    # start again (after changing the analysis)
```

laborch.build Analyses every `.aif` in `SOUND_DIRS` (about 4 minutes), saving progress to `BUILD_CACHE` after every 50 sounds, then writes `bank_v2.npz` and `bank_v2.csv`. `NAME_FIXES` maps the recording `tpt-fs3-hmo-mf` (a typo) to `tpc-fs3-hmo-mf`. `python docs/make_bank_report.py` then regenerates the report `docs/bank_v2.md` and its figure.

---

6. Recipes

A Target from a recording

```
from laborch.spectra import power_of_file
target, sus = power_of_file("my_target.wav")  # 44.1 kHz; sustain found automatically
fit = nnls_fit(bank.select(instruments=["flu", "clb", "obo"]), target)
```

Compare the RH2009 methods

```
pal = bank.select(instruments=["flu"])
t = bank.spectrum("vln-a4x-ord-mf")
for f in (dot(pal, t, normalize=True), pinv_fit(pal, t), nnls_fit(pal, t)):
    print(f.method, f.ranked(3, nonzero=False))
```

A synthetic Target (notebook 2.1, on Bank v1)

```
from laborch.synth import tone
bank1 = Bank.load(version=1)
f0 = parse_name("mrb-c3-ord-mf").freq
spec = make_fft(tone(f0, partials=[1, 4, 10], amps=[1, 0.5, 0.25]))
print(nnls_fit(bank1.select(instruments=["mrb"]), spec).summary(3))
# mrb-c3 0.657, mrb-c5 0.298, mrb-e6 0.148: a marimba C3 doubled
# on its 4th (C5) and 10th (about E6) partials
```

Try a frequency weighting (relative error per bin)

```
import numpy as np
sigma = np.maximum(target, target.max() * 1e-4)   # sigma_i proportional to T_i
fit_rel = nnls_fit(palette, target, sigma=sigma)
```

Select by family, or check the analysis of one sound

```
pal = bank.select(families=["flutes", "clarinets"], modes="ord")
row = bank.meta["vln-a4x-ord-mf"]
row["sustain_start_s"], row["sustain_end_s"], row["level_db"], row["method"]
```

Plot Target against a Recommendation

```
import matplotlib.pyplot as plt
from laborch.bank import frequencies
f = frequencies(); r = fit.top(4, refit=True)
plt.semilogy(f, target, label="Target"); plt.semilogy(f, r.model, label="Top 4")
plt.xlim(0, 5000); plt.xlabel("Hz"); plt.legend()
```

One note per instrument (manual, until Phase 2) — keep the best weight per instrument and refit:

```
best = {}
for name, w in fit.ranked():
    best.setdefault(bank.meta[name]["instrument"], name)
import numpy as np
from laborch import Palette
pal1 = Palette(list(best.values()),
               np.column_stack([bank.spectrum(n) for n in best.values()]))
print(nnls_fit(pal1, target).summary())
```

7. Data conventions

- **Sound names** — `inst-pitch-mode-dyn`, e.g. `vln-a4x-ord-mf`. Sharps as `s` (`cs4`); optional string number before the pitch (`ctb-1b4-nhr-mf`); optional variant after it (`a4o` open string, `a4x` stopped).
- **Spectra, Bank v2 (default)** — 2,048 bins, bin k at $k \times 10.77$ Hz (bin 0 = DC, last bin $\approx$ 22 kHz); **power**, averaged over the detected sustain (or the first $\leq$ 0.5 s of a struck sound); samples scaled to −1..1; each spectrum sums to the mean square of its region, so `10 log10(sum)` is its level in dBFS. Stored as float32.
- **Spectra, Bank v1** — same bins; magnitude, $\sqrt{(\Sigma|X|^2)/n}$ over the middle 70% of the note, normalized to unit sum.
- **Levels** — v2 keeps the *recorded* level, which is not the playing level: piano ff is about 4 dB quieter than pp in the McGill tracks. `Bank.load(normalize="unit")` removes levels.
- **Bank coverage** — 1,515 sounds; nearly all `mf`, with `pp` and `ff` only for piano; 29 glockenspiel notes listed in the spreadsheets were never split out (see `docs/bank_v2.md`).
- **Units of weights** — v2: a power gain on the recording (1 = as recorded). v1: a share of the fitted spectrum.

8. Testing

```
cd LabOrch
pytest            # 61 tests, about 10–15 s
pytest -v -k ravel
```

Test file	Checks
`test_names.py`	name parsing, MIDI numbers, every Bank name parses (v1 and v2), v1 unit normalization
`test_metadata.py`	families, envelope classes, a complete row per sound, variants on open strings, ranges, v2 regions close to v1, `select(families=...)`
`test_power.py`	Parseval level, powers add, sustain and decay detection on synthetic notes, fallback, given regions, rebuilding four v2 spectra from their `.aif` files, `normalize="unit"`
`test_solve.py`	NNLS recovers an exact synthetic mixture, σ weighting, `top`/refit, dB gains
`test_spectra.py`	`make_fft` rebuilds four stored v1 spectra from their `.aif` files; pure-tone peak
`test_regression_notebook_1_5.py`	violin A4 from flutes matches notebook 1.5 to three decimals; pseudo-inverse goes negative, NNLS does not
`test_synth.py`	`tone` partials, amplitudes and decay; the synthetic C3 decomposes into marimba C3 + C5 + E6; spectrum-to-audio round trip
`test_regression_ravel.py`	RH2009 *Daphnis et Chloé* example, on v1 and v2: the woodwind chord is closer to the cup mute than the straight mute

Audio tests are skipped automatically if pydub or ffmpeg is missing; data tests are skipped if the Bank folders are absent.

9. Known limitations and roadmap

Limitation	Planned in
Recorded levels are not playing levels (piano ff $\approx$ 4 dB below pp); no loudness model	Phase 2 (or new recordings)
Every frequency bin counts equally ($\sigma = 1$); with power spectra the loudest partials dominate	Phase 2 (choice of σ)
No playability: several notes may be assigned to one player	Phase 2 (one-note-per-player search)
Dynamics are mf only (except piano); 29 glockenspiel notes missing; bass sax has 8 notes	Phase 1 follow-up: re-split the glockenspiel track; source pp/ff recordings
No web interface, score export, morphing or creative mode	Phases 3–5

The full plan is in the project's Planning documents (not in the public repo).